

2 Fundamentação Teórica

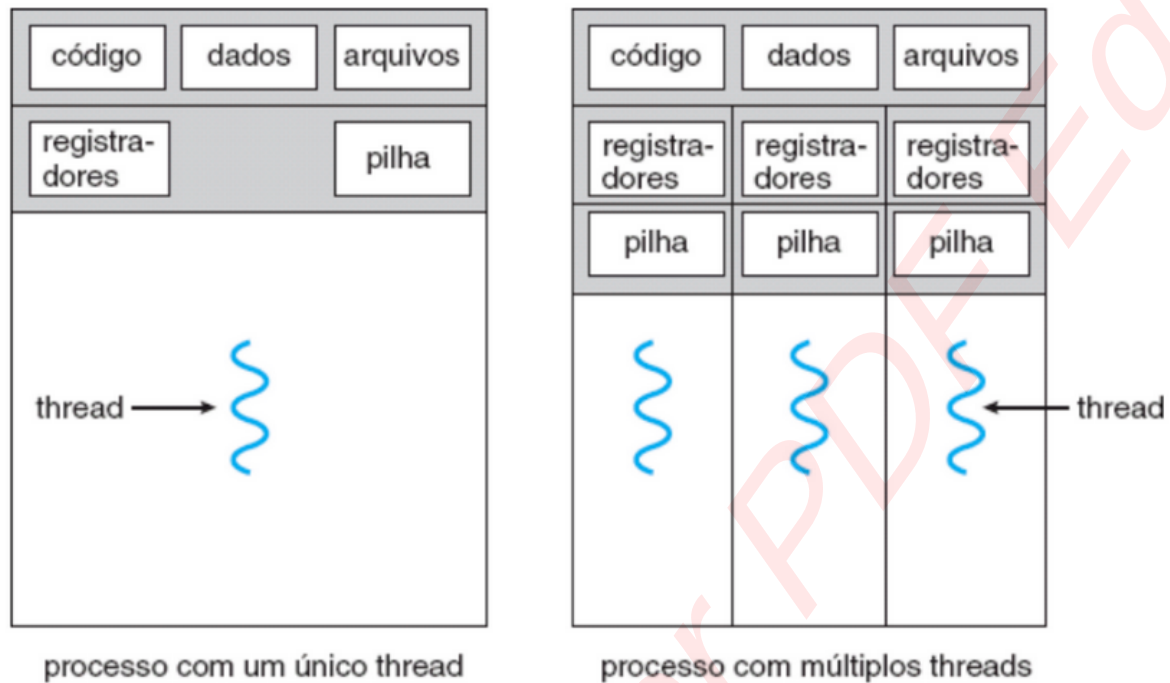
Esta seção apresenta os conceitos e teorias necessários para a compreensão do trabalho, fornecendo embasamento científico às análises e às decisões adotadas ao longo deste estudo. São abordados fundamentos de sistemas operacionais, como processos, *threads* e concorrência.

2.1 Processos e Threads

Segundo Silberschatz, Galvin e Gagne (2018), **um processo...** processo pode ser definido como um programa em execução, não se limitando apenas ao código do programa. O conceito de processo inclui também o seu estado atual durante a execução no sistema operacional, abrangendo informações como o contador de programa, registradores da CPU e áreas de memória próprias, como pilha, área de dados e *heap*. Além disso os processos podem possuir estados diferentes ao longo do seu ciclo de vida, como criação, execução, espera e término.

Uma *thread* pode ser definida como a unidade básica de utilização da CPU, sendo composta por um identificador (*thread ID*), um contador de programa, um conjunto de registradores e uma pilha. *Threads* pertencentes ao mesmo processo podem compartilhar a seção de código, a seção de dados e os recursos do sistema operacional. Todo processo possui ao menos uma *thread* de controle, quando contém múltiplas *threads*, torna-se capaz de executar mais de uma tarefa de forma **concorrente**. (Silberschatz; Galvin; Gagne, 2018) **concorrente (...).**

De acordo com Silberschatz, Galvin e Gagne (2018), o uso de múltiplas *threads* apresenta diversos benefícios, **como o aumento da capacidade de resposta**, uma vez que o uso de *threads* permite que o programa continue em execução mesmo quando parte dele está bloqueada, aumentando a interatividade com o usuário. Outro benefício é o compartilhamento de recursos, pois, diferente dos processos nos quais a comunicação ocorre por meio de memória compartilhada ou troca de mensagens, *threads* permitem o compartilhamento direto de recursos dentro do mesmo espaço de endereçamento. Além disso, o uso de *threads* proporciona economia de recursos, uma vez que a criação e o gerenciamento de *threads* demandam menor custo de memória quando comparados aos processos. Em arquiteturas multiprocessadas, *threads* podem ser executadas em paralelo em diferentes núcleos de processamento, enquanto um processo com apenas uma *thread* é limitado à execução em um único núcleo.

Figura 1 – Comparação entre processo com *single thread* e múltiplas *threads*.

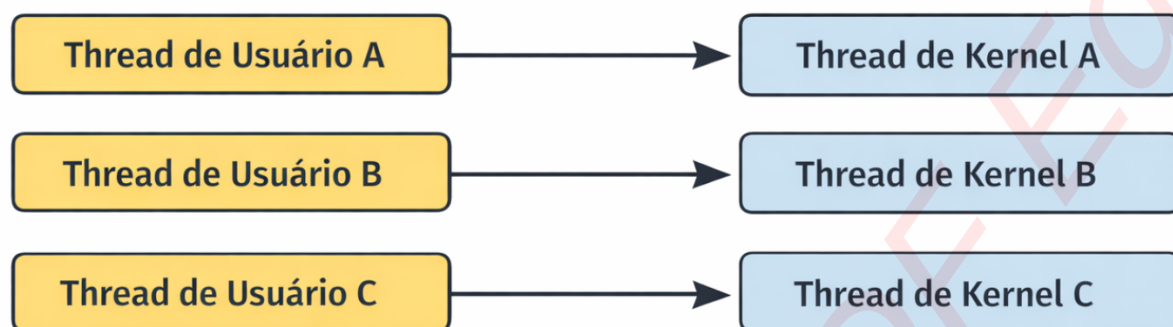
Fonte: Silberschatz, Galvin e Gagne (2018).

A Figura 1 ilustra as diferenças entre processos de *thread* única e múltiplas *threads*. Enquanto um processo com uma única *thread* possui apenas um conjunto de registradores e uma pilha, a utilização de múltiplas *threads* permite que cada *thread* mantenha seu próprio contexto de execução, compartilhando código e dados do processo. Essa comparação ajuda a compreender os benefícios de desempenho e flexibilidade trazidos pelo modelo *multithread*.

2.2 Threads Tradicionais em Java

Threads tradicionais em Java seguem o modelo de mapeamento *one-to-one* (1:1), no qual cada *thread* de nível de usuário criada pelo aplicativo é mapeada para uma *thread* de nível de *kernel*. Com isso, todas as *threads* podem acessar o *kernel* simultaneamente e serem agendadas de forma independente. Sendo assim, **ele se torna limitado**, cada *thread* adicional aumenta o peso do processo, consumindo mais recursos do sistema. Consequentemente, muitas implementações suportam um número limitado de *threads* simultâneas, imposto pelo sistema operacional. (Oracle Corporation, 2010)

Figura 2 – Modelo 1:1 (um-para-um) de mapeamento de *threads* de usuário para *threads* de *kernel*.



Fonte: Elaborado pela autora (2025).

A Figura 2 ilustra o modelo 1:1 de *threads*, onde cada *thread* de usuário é associada diretamente a uma *thread* de *kernel*.

Parágrafo confuso.

O Java fornece uma API básica de *threads* através da classe `java.lang.Thread`, isso permite que a JVM gerencie múltiplas *threads* concorrentes. A classe `Thread` implementa a interface `Runnable`, uma *thread* pode ser definida como um objeto que implementa `Runnable`. A *thread* é criada instanciando um objeto `Thread` ou passando um objeto `Runnable` ao construtor de `Thread`, a execução é iniciada pelo método `start()`, o que faz com que o método `run()` seja chamado posteriormente pela JVM. Cada *thread* passa por diferentes estados durante seu ciclo de vida, `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING` e `TERMINATED`, o que permite ao programador controlar e compreender o comportamento da *thread* durante a execução. (Oracle Corporation, 2014b)

2.3 Limitações das Threads Tradicionais

Exato! Tá aí porque você não teve tanto impacto na memória, neh? Sua thread tem pouco código e definição de variáveis. Você pode colocar essa inferência na discussão de resultados relacionando ao pouco uso de memória RAM e sugerir testes com threads mais reais em uso de recursos (dados e variáveis).

A documentação oficial da Oracle (Oracle Corporation, 2014a) define que por padrão o tamanho da pilha gerada por *thread* é aproximadamente 1 MB. Com a criação de milhares de *threads*, o consumo total de memória se torna significativo, por exemplo a criação de 10.000 *threads* de plataforma resultaria em cerca de 10 GB de memória ocupada apenas pelas pilhas, sem contar outros recursos associados a cada *thread*.

Há também o custo de criação que envolve interações diretas com o sistema operacional, o que exige chamadas de sistema (*system calls*) e a alocação de estruturas de gerenciamento de *threads* no *kernel*. Gerando assim um *overhead* maior do que a criação de objetos comuns na JVM. Ao utilizar muitas *threads* esse custo pode se tornar elevado e impactar na performance do sistema. (Oracle Corporation, 2014a)

A troca de contexto (*context switch*) também é um fator importante. De acordo com Tanenbaum e Bos (2015), o sistema operacional deve salvar o estado da *thread* atual (registradores, ponteiro de instrução, pilha, *caches*, etc.) e restaurar o estado da *thread* que

será executada em seguida. Esse processo gera *overhead*, e seu custo impacta diretamente o desempenho do sistema, contribuindo para tornar o modelo tradicional de *threads* (1:1) menos eficiente em cenários com milhares de *threads*.

O uso de *threads* tradicionais pode ser limitado em termos de escalabilidade, uma vez que o sistema operacional impõe restrições ao número de *threads* que podem ser criadas simultaneamente. No modelo *thread-per-request* onde cada tarefa cria sua própria *thread*, a abordagem torna-se ineficiente em cenários de alta concorrência, não apenas pelo consumo de memória, mas também pelo *overhead* de criação e custo de troca de contexto. Gerando o problema conhecido como C10K, que descreve a dificuldade de gerenciar mais de 10.000 conexões simultâneas utilizando *threads* tradicionais. (Kegel, 2006)

Como cada *thread* de plataforma é gerenciada pelo sistema operacional, todas competem por recursos limitados como estruturas de controle, tempo de CPU e *caches* do processador. Enquanto o número de *threads* aumenta, o *overhead* de escalonamento (*scheduling overhead*) também cresce, reduzindo a eficiência do sistema. (Tanenbaum; Bos, 2015)

sistema (...).

Verificar em todo o texto citações no fim do parágrafo. Você está colocando o ponto antes dos parênteses.

2.4 Threads Virtuais (Project Loom)

WEB é maiúsculo. Rever em todo o texto.

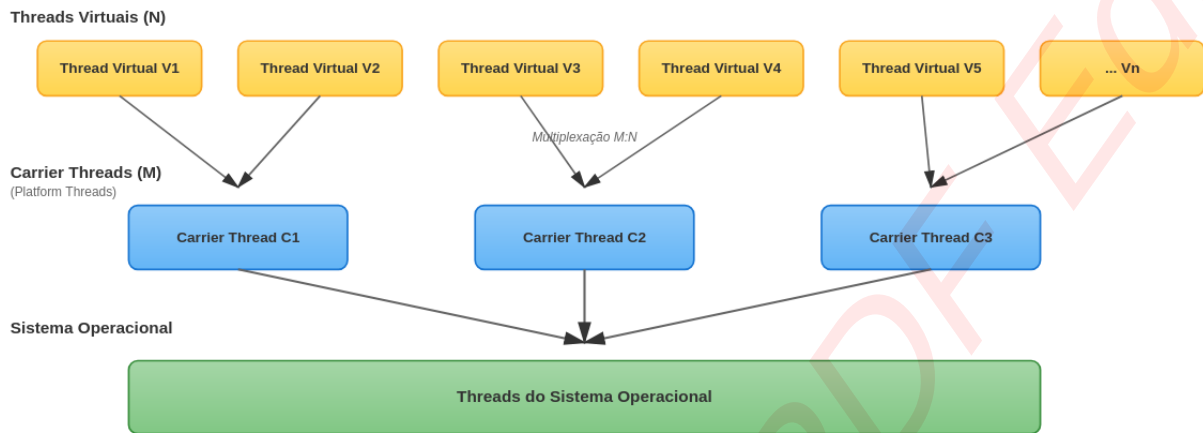
O *Project Loom* foi iniciado no final de 2017; com foco em melhorar a escalabilidade de sistemas *web* e aplicações concorrentes através do paralelismo. O modelo de *threads* tradicionais não escala bem em cenários com um grande número de tarefas simultâneas, para contornar essas limitações o projeto apresentou um novo modelo de *thread*, as *threads* virtuais. (OpenJDK Project Loom, 2020)

De acordo com a documentação do *Project Loom* OpenJDK Project Loom (2020) *threads* virtuais são *threads* leves gerenciadas pela JVM, que não estão diretamente vinculadas a *threads* do sistema operacional. Na API do Java, elas se apresentam como *threads* normais, permitindo que o programador interaja com elas de forma muito semelhante ao uso de *threads* tradicionais. A sua diferença se torna evidente ao iniciar milhares de *threads*, elas são criadas de forma eficiente, consumindo menos memória e evitando *overhead* de criação e troca de contexto.

É isso que você quer testar, então melhor rever essa frase.

Coloca que a proposta visa melhorar a eficiência de criação de milhares de threads ...

Enquanto as *threads* tradicionais utilizam o modelo 1:1, segundo a documentação oficial da Oracle Oracle Corporation (2023), as *threads* virtuais são executadas sobre um conjunto menor de *platform threads* ou *carrier threads*. Isso permite que uma grande quantidade de *threads* virtuais seja multiplexada sobre um número limitado de *threads* do sistema operacional, mecanismo análogo ao modelo M:N de multiplexação, no qual muitas *threads* lógicas (N) são associadas a menos *threads* físicas (M), melhorando assim a escalabilidade do sistema.

Figura 3 – Arquitetura de *threads* virtuais: múltiplas *threads* virtuais multiplexadas sobre *carrier threads*

Fonte: Elaborado pela autora (2025).

A Figura 3 ilustra o funcionamento das *threads* virtuais, mostrando como múltiplas *threads* lógicas podem ser multiplexadas sobre um número menor de *carrier threads*, de forma análoga ao modelo M:N.

Há dois conceitos, *mounting* e *unmounting*, que descrevem como uma *thread* virtual é associada a uma *carrier thread* para executar suas instruções. Quando a *thread* encontra uma operação bloqueante, como operações de I/O (*sockets*, arquivos), `Thread.sleep()` ou `Object.wait()`, ela é desmontada da *carrier thread*, liberando esta última para executar outras *threads* virtuais. Após a conclusão da operação bloqueante, a *thread* virtual é remontada em uma *carrier thread* disponível, retomando sua execução. Assim esse mecanismo permite a execução de um grande número de *threads* virtuais sobre um número limitado de *carrier threads*, aumentando a escalabilidade e reduzindo o consumo de recursos do sistema. Oracle Corporation (2023)

Na documentação Oracle Corporation (2014b) também é descrito que as *threads* virtuais possuem sua pilha (*stack*) alocada dinamicamente no *heap* da JVM, enquanto as *threads* tradicionais possuem sua pilha fixa e gerenciada pelo sistema operacional. Essa pilha dinâmica cresce e reduz conforme a necessidade, com tamanho inicial muito menor aproximadamente 1 KB.

Podemos observar que *threads* virtuais são adequadas para *workloads I/O-bound*, onde a maior parte do tempo é gasta aguardando operações de entrada/saída. Em tarefas *CPU-bound*, o benefício é limitado, já que o gargalo ainda será o número de *cores* disponíveis.